CSE105: Linux & Shell Programming

Comprehensive Question Bank, Lab Practicals & Solved Scenarios

Department of Computer Science & Engineering • Academic Board

Edition 2026–2027

Contents

1 Unit 1: Linux Basics & OS Architecture Question Bank **1**
- 1.1 Multiple Choice Questions (MCQs) . . . 1
- 1.2 Technical Viva-Voce Questions & Conceptual Answers . . . 2
- 1.3 Lab Practical Exercises & Step-by-Step CLI Execution . . . 3

2 Unit 2: System Utilities & Basic CLI Commands Question Bank **4**
- 2.1 Multiple Choice Questions (MCQs) . . . 4
- 2.2 Technical Viva-Voce Questions & Conceptual Answers . . . 5
- 2.3 Lab Practical Exercises & Step-by-Step CLI Execution . . . 6

3 Unit 3: Filesystem Hierarchy & File Operations Question Bank **7**
- 3.1 Multiple Choice Questions (MCQs) . . . 7
- 3.2 Technical Viva-Voce Questions & Conceptual Answers . . . 8
- 3.3 Lab Practical Exercises & Step-by-Step CLI Execution . . . 9

4 Unit 4: File Permissions & Ownership Administration Question Bank **10**
- 4.1 Multiple Choice Questions (MCQs) . . . 10
- 4.2 Technical Viva-Voce Questions & Conceptual Answers . . . 11
- 4.3 Lab Practical Exercises & Step-by-Step CLI Execution . . . 12

5 Unit 5: Shell Features, Redirection & Pipelines Question Bank **13**
- 5.1 Multiple Choice Questions (MCQs) . . . 13
- 5.2 Technical Viva-Voce Questions & Conceptual Answers . . . 14
- 5.3 Lab Practical Exercises & Step-by-Step CLI Execution . . . 15

6 Unit 6: Filters & Regular Expressions Question Bank **16**
- 6.1 Multiple Choice Questions (MCQs) . . . 16
- 6.2 Technical Viva-Voce Questions & Conceptual Answers . . . 17
- 6.3 Lab Practical Exercises & Step-by-Step CLI Execution . . . 18

7 Unit 7: User & Group Administration Question Bank **19**
- 7.1 Multiple Choice Questions (MCQs) . . . 19
- 7.2 Technical Viva-Voce Questions & Conceptual Answers . . . 20
- 7.3 Lab Practical Exercises & Step-by-Step CLI Execution . . . 21

8 Unit 8: Process Management & Signal Architecture Question Bank **22**
- 8.1 Multiple Choice Questions (MCQs) . . . 22
- 8.2 Technical Viva-Voce Questions & Conceptual Answers . . . 23
- 8.3 Lab Practical Exercises & Step-by-Step CLI Execution . . . 24

9 Unit 9: vi/vim Modal Text Editor Question Bank **25**
- 9.1 Multiple Choice Questions (MCQs) . . . 25
- 9.2 Technical Viva-Voce Questions & Conceptual Answers . . . 26

10 Unit 10: Shell Programming & Automation Question Bank **27**
- 10.1 Multiple Choice Questions (MCQs) . . . 27
- 10.2 Shell Scripting & DevOps Automation Challenges . . . 28

11 Unit 11: Development Tools & Git Version Control Question Bank **30**
11.1 Multiple Choice Questions (MCQs) 30
11.2 Technical Viva-Voce Questions & Conceptual Answers 31
11.3 Lab Practical Exercises & Step-by-Step CLI Execution 32

Chapter 1

Unit 1: Linux Basics & OS Architecture Question Bank

1.1 Multiple Choice Questions (MCQs)

Q1.1. Which of the following components in a Linux system executes directly in privileged CPU Ring 0?

(a) The GNU C Library (glibc)
(b) The Linux Monolithic Kernel
(c) The Bourne-Again Shell (bash)
(d) The X11 Windowing Server

Q1.2. The system call interface serves primarily as:

(a) A compiler optimization pipeline for user applications
(b) The controlled boundary through which user-space processes invoke kernel services
(c) A direct communication protocol between web browsers and network routers
(d) The bootloader mechanism responsible for loading the kernel image into RAM

Q1.3. Which Linux distribution uses the DNF/RPM package management framework?

(a) Ubuntu
(b) Arch Linux
(c) Red Hat Enterprise Linux (RHEL)
(d) Alpine Linux

Q1.4. Under the modern UEFI specification, what filesystem format is strictly required for the EFI System Partition (ESP)?

(a) ext4
(b) NTFS
(c) FAT32
(d) Btrfs

Q1.5. What is the primary architectural improvement of WSL 2 compared to WSL 1?

(a) WSL 2 translates system calls to Windows NT APIs directly
(b) WSL 2 incorporates a complete, real Linux kernel running in a lightweight Hyper-V VM
(c) WSL 2 eliminates the need for any hypervisor virtualization extensions
(d) WSL 2 operates strictly as a remote SSH client to cloud instances

Q1.6. Which display server architecture eliminates tear artifacts by integrating compositor and window manager functions?

(a) X11
(b) Wayland
(c) Xorg
(d) Mir

Q1.7. Which hypervisor type runs directly on bare-metal physical hardware without an underlying

host OS?

(a) Type-1 Bare-Metal Hypervisor
(b) Type-2 Hosted Hypervisor
(c) VirtualBox
(d) VMware Workstation

Q1.8. Why must Windows Fast Startup be disabled when configuring an Ubuntu dual-boot system?

(a) Fast Startup permanently disables UEFI secure boot
(b) Fast Startup leaves Windows NTFS partitions in a locked hibernation state that causes corruption if mounted in Linux
(c) Fast Startup prevents the network card from acquiring an IP address in Linux
(d) Fast Startup deletes the GRUB bootloader partition on reboot

Q1.9. What is the fundamental philosophy of Unix composability popularized by Doug McIlroy?

(a) Monolithic applications handling all user requests simultaneously
(b) Programs that do one thing well and communicate through universal text streams
(c) Proprietary binary protocols between user applications
(d) Executing all processes with superuser root privileges

Q1.10. Which partition acts as virtual memory overflow when physical RAM is fully saturated?

(a) `/boot`
(b) `/dev`
(c) `swap`
(d) `/etc`

Table 1.1: **Unit 1 MCQ Answer Key & Verification**

Question	Answer	Conceptual Core
1.1	**(b)**	The Linux kernel operates in CPU Ring 0 (supervisor mode); applications run in Ring 3.
1.2	**(b)**	System calls provide the formal gateway transitioning execution from user to kernel mode.
1.3	**(c)**	RHEL and Fedora utilize the RPM package format with DNF/YUM package managers.
1.4	**(c)**	UEFI firmware requires the ESP partition to be formatted as FAT32/FAT16.
1.5	**(b)**	WSL 2 runs a customized real Linux kernel in a lightweight Hyper-V utility VM.
1.6	**(b)**	Wayland eliminates tearing by ensuring every frame rendered by clients is fully composited.
1.7	**(a)**	Type-1 hypervisors (Xen, ESXi, KVM) execute directly on physical silicon.
1.8	**(b)**	Fast Startup locks NTFS filesystem metadata; write operations from Linux cause data loss.
1.9	**(b)**	Unix modularity: small, focused utilities composable via standard text pipes.
1.10	**(c)**	The swap partition or swap file holds inactive memory pages paged out of physical RAM.

1.2 Technical Viva-Voce Questions & Conceptual Answers

Q1. Explain the architectural boundary between User Space and Kernel Space in Linux.

The CPU enforces hardware privilege rings (Ring 0 to Ring 3 on x86). Ring 0 represents **Kernel Space**, wherein the operating system kernel executes with unrestricted access to physical memory, CPU control registers, and raw peripheral hardware. Ring 3 represents **User Space**, wherein unprivileged user applications, shells, and daemons run in sandboxed virtual address spaces. Whenever an application needs disk access, network transmission, or memory allocation, it must execute a hardware trap or software interrupt (via system calls like `read()` or `write()`), switching CPU context to Ring 0.

Q2. Differentiate between Type-1 and Type-2 Hypervisors in virtualization.

Type-1 Hypervisors (Bare-Metal): Run directly upon the physical host hardware without an intermediate operating system (e.g., VMware ESXi, Proxmox VE, KVM). They deliver near-native performance and lower latency. **Type-2 Hypervisors (Hosted):** Run as software applications atop an existing host operating system (e.g., Oracle VM VirtualBox, VMware Workstation). Hardware requests from guest operating systems traverse both the hypervisor and the host OS kernel, introducing minor virtualization overhead.

1.3 Lab Practical Exercises & Step-by-Step CLI Execution

Lab Practical Exercise 1.1 — Hardware Architecture & Virtualization Diagnostics [Hands-On Implementation]

Objective: Interrogate the underlying hardware CPU architecture, inspect whether hardware virtualization extensions (VT-x/AMD-V) are active, and evaluate kernel release specifications.

```
$ Interactive Terminal Verification

$ uname -srm
Linux 6.8.0-45-generic x86_64

$ lscpu | grep -E "Model name|Virtualization|Architecture"
Architecture:             x86_64
Model name:               Intel(R) Core(TM) i7-12700H
Virtualization:           VT-x

$ grep -E -c "(vmx|svm)" /proc/cpuinfo
20
```

Technical Verification: The return of `VT-x` from `lscpu` and a count > 0 from `/proc/cpuinfo` confirms that Intel VT-x hardware-assisted virtualization is enabled in UEFI firmware, allowing VirtualBox or WSL 2 to operate.

Chapter 2

Unit 2: System Utilities & Basic CLI Commands Question Bank

2.1 Multiple Choice Questions (MCQs)

Q2.1. What does an exit status code of 0 ($?) indicate in Linux CLI grammar?

(a) Fatal error occurred
(b) Successful command execution without error
(c) Command timed out waiting for input
(d) Process was killed by signal SIGKILL

Q2.2. Which switch enables the interpretation of backslash escapes (such as \n and \t) in the echo command?

(a) `-n`
(b) `-e`
(c) `-E`
(d) `-s`

Q2.3. What is the output of the command: expr 15 / 4?

(a) `3.75`
(b) `3`
(c) `4`
(d) `Error: floating point not supported`

Q2.4. How is decimal precision configured when using the bc arbitrary-precision calculator?

(a) `set precision=N`
(b) `scale=N`
(c) `float=N`
(d) `digits=N`

Q2.5. Which command records a complete terminal session into a typescript log file?

(a) `record`
(b) `history`
(c) `script`
(d) `logger`

Q2.6. Which manual page section contains configuration file formats (such as /etc/passwd and /etc/shadow)?

(a) Section 1
(b) Section 2
(c) Section 4
(d) Section 5

Q2.7. What do the three floating-point numbers in the output of the uptime command represent?

(a) CPU core temperatures

(b) System load averages over the past 1, 5, and 15 minutes
(c) RAM utilization percentages
(d) Network packet transmission rates

Q2.8. Which command suppresses character echoing on the terminal, making it ideal for hidden password prompts?

(a) `stty -echo`
(b) `tty -s`
(c) `passwd -q`
(d) `echo -h`

Q2.9. Which format string passed to the date command emits the Unix Epoch timestamp?

(a) `+%Y`
(b) `+%s`
(c) `+%T`
(d) `+%z`

Q2.10. What is the function of the apropos command?

(a) Displays real-time memory distribution
(b) Searches manual page short descriptions for a given keyword string
(c) Compiles a C program into machine bytecode
(d) Reboots the operating system

Table 2.1: **Unit 2 MCQ Answer Key & Verification**

Question	Answer	Conceptual Core
2.1	**(b)**	POSIX exit status 0 indicates success; non-zero (1–255) indicates an error condition.
2.2	**(b)**	`echo -e` enables backslash escape character interpretation.
2.3	**(b)**	`expr` performs integer arithmetic exclusively, truncating fractional remainders ($15/4 = 3$).
2.4	**(b)**	The `scale=N` variable in `bc` dictates the number of fractional decimal digits.
2.5	**(c)**	The `script` command spawns a pseudo-terminal capturing all session I/O.
2.6	**(d)**	Section 5 of the manual system is reserved for File Formats and Conventions.
2.7	**(b)**	Uptime load averages represent the average number of runnable processes over 1, 5, 15 min.
2.8	**(a)**	`stty -echo` instructs the terminal line discipline to disable character echoing.
2.9	**(b)**	`+%s` emits seconds elapsed since 1970-01-01 00:00:00 UTC (Unix Epoch).
2.10	**(b)**	`apropos` (equivalent to `man -k`) searches man page database descriptions.

2.2 Technical Viva-Voce Questions & Conceptual Answers

Q1. Explain why expr 5 * 4 produces a syntax error in Bash, and how to fix it.

In Bash, the asterisk (`*`) is a reserved wildcard globbing character that expands to all filenames in the active working directory. When the shell encounters `expr 5 * 4`, it expands `*` into a list of filenames before invoking `expr`, resulting in an invalid argument list (e.g., `expr 5 file1.txt file2.txt 4`). To perform multiplication, the asterisk must be escaped or quoted: `expr 5 \* 4` or `expr 5 '*' 4`.

Q2. Explain the purpose of the script and scriptreplay commands.

The `script` utility records terminal sessions for auditing, debugging, and instructional replay. By invoking `script -t=timing.log session.log`, the utility captures both keystrokes and precise millisecond timing. Later, executing `scriptreplay timing.log session.log` replays the interactive terminal session at exact original speeds.

2.3 Lab Practical Exercises & Step-by-Step CLI Execution

Lab Practical Exercise 2.1 — Precision Arithmetic and System Identity Audit [Hands-On Implementation]

Objective: Calculate mathematical constants to high decimal precision and generate a formatted system diagnostic report.

$ Interactive Terminal Verification

```
# 1. Arbitrary precision calculation of Pi via Ramanujan formula approximation:
$ echo "scale=8; 355 / 113" | bc
3.14159292

# 2. Formatted system status header via printf:
$ printf "+----------------------+-------------------------------------+\n"
$ printf "| %-20s | %-35s |\n" "Host System" "$(uname -n)" "Kernel" "$(uname -r)" "Uptime" "$(
    uptime -p)"
$ printf "+----------------------+-------------------------------------+\n"
+----------------------+-------------------------------------+
| Host System          | workstation-dev-01                  |
| Kernel               | 6.8.0-45-generic                    |
| Uptime               | up 4 days, 6 hours                  |
+----------------------+-------------------------------------+
```

Chapter 3

Unit 3: Filesystem Hierarchy & File Operations Question Bank

3.1 Multiple Choice Questions (MCQs)

Q3.1. Which directory in the Filesystem Hierarchy Standard contains device nodes (such as /dev/sda and /dev/null)?

(a) `/etc`
(b) `/dev`
(c) `/sys`
(d) `/proc`

Q3.2. What metadata component is NOT stored inside a file's inode in an ext4 filesystem?

(a) The file's byte size
(b) The file's name
(c) The owner User ID (UID)
(d) The access timestamps

Q3.3. What happens to a file's data blocks when one of its two hard links is deleted using rm?

(a) The data blocks are immediately erased and returned to the free block pool
(b) The data remains untouched; only the inode's link count is decremented from 2 to 1
(c) The remaining hard link becomes a broken link pointing to NULL
(d) The filesystem marks the inode as corrupted

Q3.4. Why cannot a hard link be established across two distinct physical disk partitions?

(a) Partition tables do not allow file sharing
(b) Inode numbers are unique only within a single filesystem instance
(c) Hard links require identical disk sector block sizes
(d) The root user alone holds permission to create cross-partition links

Q3.5. Which command switch ensures mkdir creates parent directories as needed without throwing errors?

(a) `-r`
(b) `-p`
(c) `-f`
(d) `-a`

Q3.6. Which command locates regular files modified within the past 48 hours and larger than 100 Megabytes?

(a) `find / -type f -mtime -2 -size +100M`
(b) `search -f -time 48h -size 100M`
(c) `ls -la –size 100M –time 48h`
(d) `locate -type f -size +100M`

Q3.7. What is the difference between du and df commands?

(a) `df` measures directory sizes; `du` displays disk device mount points
(b) `df` inspects filesystem-level superblock allocation; `du` traverses file directory trees
(c) `du` requires root access; `df` is restricted to guest users
(d) `du` and `df` are identical aliases

Q3.8. In Bash history expansion, what does the token !! execute?
(a) Repeats command number 1 in history
(b) Repeats the immediately preceding command
(c) Clears the history buffer
(d) Executes the command matching string "!"

Q3.9. What does the file command do when executed against an arbitrary file?
(a) Displays file permissions in octal format
(b) Examines file magic bytes to report the true MIME type and format
(c) Calculates the SHA-256 hash of the file
(d) Creates an empty file

Q3.10. Which tree command option restricts directory traversal to a depth of 2 levels?
(a) `tree -d 2`
(b) `tree -L 2`
(c) `tree -depth 2`
(d) `tree -m 2`

Table 3.1: **Unit 3 MCQ Answer Key & Verification**

Question	Answer	Conceptual Core
3.1	**(b)**	`/dev` houses special device nodes representing physical and pseudo hardware.
3.2	**(b)**	The file name is stored in the parent directory entry table, **not** in the inode itself.
3.3	**(b)**	Deleting a hard link decrements the link counter; data is deallocated only when links = 0.
3.4	**(b)**	Inodes are internal table indices unique to a specific physical filesystem partition.
3.5	**(b)**	`mkdir -p` creates intermediary parent directories recursively without failure.
3.6	**(a)**	`find / -type f -mtime -2 -size +100M` matches regular files, modified past 2 days, > 100MB.
3.7	**(b)**	`df` queries superblock allocations; `du` sums actual file tree disk usage.
3.8	**(b)**	`!!` repeats the last executed command line.
3.9	**(b)**	`file` reads magic numbers in the header to identify executable formats, encodings, etc.
3.10	**(b)**	`tree -L 2` caps recursive tree exploration at a depth limit of 2 levels.

3.2 Technical Viva-Voce Questions & Conceptual Answers

Q1. Explain why files in /proc display a byte size of 0 yet yield text when read with cat.

The `/proc` directory is a virtual pseudo-filesystem instantiated directly in RAM by the Linux kernel. It does not exist on physical disk storage. When an application reads a file in `/proc` (such as `/proc/cpuinfo` or `/proc/meminfo`), the kernel dynamically intercepts the read system call, formats internal kernel data structures into ASCII text, and streams it back to user space. Because no physical disk sectors are occupied,

the filesystem reports their static size as 0 bytes.

Q2. Contrast Hard Links and Symbolic (Soft) Links.

- **Hard Link:** A directory entry pointing to an existing inode number. It shares identical permissions, timestamps, and data blocks. Deleting the original file does not destroy data as long as at least one hard link remains. Hard links cannot span across filesystems and cannot point to directories.
- **Symbolic Link:** A distinct file with its own inode and type `l`. Its data block contains the text pathname of the target. If the target file is deleted, the symlink becomes "dangling" or broken. Symlinks can cross partition boundaries and link directories.

3.3 Lab Practical Exercises & Step-by-Step CLI Execution

Lab Practical Exercise 3.1 — Inode Inspection and Link Lifecycle Demonstration [Hands-On Implementation]

Objective: Trace inode allocation across hard links and symbolic links, verifying link counts and file survival after target removal.

```
$ Interactive Terminal Verification

# 1. Create file and inspect its inode number:
$ echo "Mission Critical Data" > original.txt
$ ls -li original.txt
1048578 -rw-r--r-- 1 student student 22 Oct 22 08:00 original.txt

# 2. Create hard link and symbolic link:
$ ln original.txt hardlink.txt
$ ln -s original.txt symlink.txt

# 3. Inspect inode numbers and link counts:
$ ls -li original.txt hardlink.txt symlink.txt
1048578 -rw-r--r-- 2 student student 22 Oct 22 08:00 hardlink.txt
1048578 -rw-r--r-- 2 student student 22 Oct 22 08:00 original.txt
1048579 lrwxrwxrwx 1 student student 12 Oct 22 08:01 symlink.txt -> original.txt

# 4. Remove original file and test data availability:
$ rm original.txt
$ cat hardlink.txt
Mission Critical Data
$ cat symlink.txt
cat: symlink.txt: No such file or directory
```

Conclusion: `hardlink.txt` retains access to data because inode `1048578` still possesses a link count of 1. `symlink.txt` fails because its target path is absent.

Chapter 4

Unit 4: File Permissions & Ownership Administration Question Bank

4.1 Multiple Choice Questions (MCQs)

Q4.1. What does execute (x) permission grant when assigned to a DIRECTORY in Linux?

(a) The ability to compile scripts located within the directory
(b) The ability to enter / traverse the directory via cd and access its file inodes
(c) The permission to delete the directory
(d) The right to modify directory names

Q4.2. What is the numeric octal representation of the permission string: -rwxr-xr–?

(a) `755`
(b) `754`
(c) `764`
(d) `654`

Q4.3. If the system umask is configured as 027, what permissions will a newly created regular file possess?

(a) `640` (`-rw-r--`)
(b) `750` (`-rwxr-x–`)
(c) `644` (`-rw-r-r-`)
(d) `600` (`-rw——-`)

Q4.4. What is the effect of setting the Set User ID (SUID) bit on an executable binary?

(a) The program executes with elevated priority in the Completely Fair Scheduler
(b) Any user running the binary temporarily assumes the privileges of the file's owner
(c) The file is encrypted against unauthorized modification
(d) The program is automatically restarted upon system reboot

Q4.5. Which special permission bit prevents regular users from deleting each other's files within the /tmp shared directory?

(a) SUID bit (4000)
(b) SGID bit (2000)
(c) Sticky Bit (1000)
(d) Immutable bit (+i)

Q4.6. Which command recursively assigns user owner 'www-data' and group owner 'www-data' to /var/www/html?

(a) `chmod -R 777 /var/www/html`
(b) `chown -R www-data:www-data /var/www/html`
(c) `chgrp -R www-data /var/www/html`
(d) `usermod -aG www-data /var/www/html`

Q4.7. What catastrophic error occurs if an administrator executes: usermod -G devops student instead of usermod -aG devops student?

(a) The user's password is deleted
(b) The user is stripped from all their other supplementary groups (including sudo)
(c) The user's home directory is permanently deleted
(d) The command fails with a syntax error

Q4.8. How is the Sticky Bit visually denoted in the long listing (ls -l) permission string?

(a) A letter `s` in the owner execute position
(b) A letter `s` in the group execute position
(c) A letter `t` in the others execute position
(d) A letter `x` in all positions

Q4.9. Which octal value corresponds to assigning read and execute permissions only to group and others, while giving full control to the owner?

(a) `755`
(b) `744`
(c) `766`
(d) `775`

Q4.10. Which command audits the entire filesystem for unauthorized SUID root executable binaries?

(a) `find / -perm /4000 -user root -type f 2>/dev/null`
(b) `ls -la /sbin | grep suid`
(c) `ps aux | grep root`
(d) `check-suid -all`

Table 4.1: **Unit 4 MCQ Answer Key & Verification**

Question	Answer	Conceptual Core
4.1	**(b)**	Execute on a directory grants the right to traverse into it and resolve child inodes.
4.2	**(b)**	`rwx`=7, `r-x`=5, `r--`=4 → `754`.
4.3	**(a)**	Base file $666 - 027 =$ **640** (`rw-r--`). Files never get execute by default.
4.4	**(b)**	SUID executes the binary under the security context of the binary's owner (e.g. root).
4.5	**(c)**	The Sticky bit (`1000`) restricts deletion in shared directories to file owners and root.
4.6	**(b)**	`chown -R owner:group path` recursively alters user and group ownership.
4.7	**(b)**	Omitting `-a` (append) with `-G` overwrites existing supplementary group memberships.
4.8	**(c)**	Sticky bit appears as `t` (or `T` if execute is absent) in others triad (`drwxrwxrwt`).
4.9	**(a)**	Owner: 7 (`rwx`), Group: 5 (`r-x`), Others: 5 (`r-x`) → `755`.
4.10	**(a)**	`find / -perm /4000` locates files with SUID bit set; `2>/dev/null` silences errors.

4.2 Technical Viva-Voce Questions & Conceptual Answers

Q1. Explain the Directory Deletion Paradox in Unix permissions.

In Unix, the permission to delete or rename a file is governed exclusively by the **write and execute** permissions on the **containing parent directory**, not the permissions on the file itself. A directory is

essentially an index table mapping names to inode numbers. Deleting a file modifies the parent directory's table. Consequently, an ordinary user who has write access to a folder can delete a file owned by `root` with permissions `r-r-r-`, because the operation simply alters the directory entry.

Q2. Explain the difference between umask 022, 027, and 077.

- `umask 022`: Files created with mode `644` (`rw-r-r-`), dirs with `755` (`rwxr-xr-x`). World-readable default.
- `umask 027`: Files created with mode `640` (`rw-r--`), dirs with `750` (`rwxr-x-`). Group can read; others have zero permissions.
- `umask 077`: Files created with mode `600` (`rw---`), dirs with `700` (`rwx---`). Strict privacy; only the owner can read or enter.

4.3 Lab Practical Exercises & Step-by-Step CLI Execution

Lab Practical Exercise 4.1 — SGID Collaboration Folder Setup [Hands-On Implementation]

Objective: Configure a shared collaboration directory for a team where all newly created files automatically inherit the parent group ownership.

$ Interactive Terminal Verification

```
# 1. Create team group and directory:
$ sudo groupadd engineers
$ sudo usermod -aG engineers rahul
$ sudo mkdir /opt/collab
$ sudo chown :engineers /opt/collab

# 2. Assign read/write/execute and SGID bit (octal 2775):
$ sudo chmod 2775 /opt/collab

# 3. Verify directory permissions:
$ ls -ld /opt/collab
drwxrwsr-x 2 root engineers 4096 Oct 22 08:15 /opt/collab

# 4. Create a test file as user rahul:
$ touch /opt/collab/spec.pdf
$ ls -l /opt/collab/spec.pdf
-rw-r--r-- 1 rahul engineers 0 Oct 22 08:16 /opt/collab/spec.pdf
```

Verification: `spec.pdf` automatically inherits the group `engineers` rather than Rahul's primary group, enabling collaborative editing.

Chapter 5

Unit 5: Shell Features, Redirection & Pipelines Question Bank

5.1 Multiple Choice Questions (MCQs)

Q5.1. Which file descriptor integer is assigned to Standard Error (stderr) by POSIX operating systems?

(a) `0`
(b) `1`
(c) `2`
(d) `3`

Q5.2. What does the operator » file accomplish in a shell command line?

(a) Overwrites target file with standard error
(b) Appends standard output stream to the end of target file
(c) Redirects standard input from target file
(d) Pipes output to background daemon

Q5.3. Which shell redirection operator feeds multi-line text blocks directly to a command's standard input until a delimiter token is encountered?

(a) Here-String («<)
(b) Here-Document («)
(c) Append Redirection (»)
(d) Input Redirection (<)

Q5.4. What occurs when executing: command > out.log 2>&1?

(a) Both stdout and stderr are merged and written into out.log
(b) stdout is written to out.log, while stderr is discarded
(c) A syntax error is raised by the shell parser
(d) stderr is written to out.log, but stdout is echoed to the screen

Q5.5. Which command duplicates a pipeline stream, writing it to a disk file while concurrently passing it down the pipeline?

(a) `split`
(b) `tee`
(c) `fork`
(d) `clone`

Q5.6. In wildcard globbing, what does the pattern report[!0-9].txt match?

(a) Files named report followed by any decimal digit
(b) Files named report followed by any single character that is NOT a digit
(c) Files named report0.txt through report9.txt
(d) Any file starting with report

Q5.7. What is the default exit status of a multi-stage shell pipeline cmd1 | cmd2 | cmd3?

(a) The exit status of cmd1
(b) The exit status of cmd2
(c) The exit status of the final command, cmd3
(d) The arithmetic bitwise OR of all exit statuses

Q5.8. How can a Bash script be instructed to fail if ANY command in a pipeline fails?
(a) `set -e`
(b) `set -o pipefail`
(c) `set -u`
(d) `shopt -s failglob`

Q5.9. Where is data discarded when redirected via: command 2> /dev/null?
(a) To a volatile RAM disk
(b) To the Linux kernel bit-bucket special character device, silently discarded
(c) To the root user's mailbox
(d) To the swap partition

Q5.10. Which wildcard pattern matches files starting with 'app', followed by exactly two arbitrary characters, ending with '.log'?
(a) `app*.log`
(b) `app??.log`
(c) `app[2].log`
(d) `app{2}.log`

Table 5.1: **Unit 5 MCQ Answer Key & Verification**

Question	Answer	Conceptual Core
5.1	**(c)**	Standard Error is assigned File Descriptor 2 (`stderr`).
5.2	**(b)**	» appends `stdout` without truncating existing file content.
5.3	**(b)**	« `DELIMITER` is a Here-Document construct.
5.4	**(a)**	`> out.log 2>&1` redirects stdout to `out.log` and duplicates FD 2 to FD 1.
5.5	**(b)**	`tee` reads from stdin and writes concurrently to stdout and files.
5.6	**(b)**	`[!0-9]` matches any character not in the range 0–9.
5.7	**(c)**	By default, a pipeline returns the exit status of the right-most command.
5.8	**(b)**	`set -o pipefail` ensures the pipeline return status reflects any failure.
5.9	**(b)**	`/dev/null` destroys all data written to it immediately.
5.10	**(b)**	Each `?` matches exactly one single character (`app??.log` matches 2 chars).

5.2 Technical Viva-Voce Questions & Conceptual Answers

Q1. Explain why command 2>&1 > file does NOT merge streams into the file.

The shell processes redirection operators strictly from left to right:

1. When the shell encounters `2>&1`, descriptor 2 (`stderr`) is duplicated to point wherever descriptor 1 (`stdout`) currently points (which is the active display terminal).
2. Next, when the shell encounters `> file`, descriptor 1 (`stdout`) is redirected to point to the file.

As a result, `stderr` continues pointing to the terminal, and only `stdout` is redirected into `file`. The correct order is `> file 2>&1`.

Q2. Describe the operational mechanics of an anonymous kernel pipe (|).

When the shell executes `cmd1 | cmd2`, it requests the kernel create an anonymous pipe via the `pipe()` system call, yielding two file descriptors: a write end and a read end. The shell forks two child processes: the write end is duplicated to `cmd1`'s stdout, and the read end is duplicated to `cmd2`'s stdin. Both processes run concurrently. Data flow is buffered in kernel memory (typically 64 KB). If `cmd1` generates data faster than `cmd2` consumes it, `cmd1` is paused by the kernel scheduler until buffer space frees up.

5.3 Lab Practical Exercises & Step-by-Step CLI Execution

Lab Practical Exercise 5.1 — Diagnostic Log Tapping Pipeline [Hands-On Implementation]

Objective: Build a multi-stage pipeline that searches configuration files, segregates errors to an error log, writes clean results to disk, and prints the total count to the console.

```
$ Interactive Terminal Verification

$ find /etc -name "*.conf" 2> find_errors.log | tee conf_inventory.txt | wc -l
148

$ head -n 3 conf_inventory.txt
/etc/resolv.conf
/etc/sysctl.conf
/etc/fuse.conf

$ head -n 3 find_errors.log
find: '/etc/ssl/private': Permission denied
```

Chapter 6

Unit 6: Filters & Regular Expressions Question Bank

6.1 Multiple Choice Questions (MCQs)

Q6.1. What does the regular expression anchor ^ (caret) match?

(a) The literal character ^
(b) The end of a line
(c) The beginning of a line
(d) Any single alphanumeric character

Q6.2. Which grep command switch enables Extended Regular Expressions (ERE)?

(a) `-e`
(b) `-E`
(c) `-F`
(d) `-x`

Q6.3. What is the prerequisite for the uniq command to successfully eliminate duplicate lines?

(a) The input must be filtered through cut
(b) The input lines must be sorted and immediately adjacent
(c) The file must be smaller than 10 Megabytes
(d) The command must be run with root privileges

Q6.4. Which command extracts fields 1 and 3 from a colon-delimited text file?

(a) `cut -d":" -f1,3 file.txt`
(b) `grep -f 1,3 file.txt`
(c) `tr -d ":" file.txt`
(d) `sort -f 1,3 file.txt`

Q6.5. What is the output of the command: echo "hello world" | tr 'a-z' 'A-Z'?

(a) `hello world`
(b) `HELLO WORLD`
(c) `hELLO wORLD`
(d) `Hello World`

Q6.6. Which regular expression quantifier matches zero or one occurrence of the preceding element in ERE?

(a) `*`
(b) `+`
(c) `?`
(d) `{1}`

Q6.7. Which tail command option activates live streaming follow-mode to monitor expanding log files in real-time?

(a) `-s`

(b) `-f`
(c) `-r`
(d) `-l`

Q6.8. Which command sorts a text file numerically on the second column in descending order using comma delimiters?

(a) `sort -n -r -k2 -t"," file.csv`
(b) `order -d 2 -n file.csv`
(c) `cut -k2 file.csv | sort -r`
(d) `sort -descending -col 2 file.csv`

Q6.9. What does the grep switch -v do?

(a) Prints verbose debugging output
(b) Inverts matching: outputs only lines that do NOT match the pattern
(c) Shows matching version numbers
(d) Counts total character volume

Q6.10. Which POSIX regular expression matches completely blank lines?

(a) `^.*$`
(b) `^$`
(c) `\n`
(d) `[^]`

Table 6.1: **Unit 6 MCQ Answer Key & Verification**

Question	Answer	Conceptual Core
6.1	**(c)**	Caret (^) asserts position at the start of a line.
6.2	**(b)**	`grep -E` activates Extended Regular Expressions (equivalent to `egrep`).
6.3	**(b)**	`uniq` compares adjacent lines only; hence prior `sort` is mandatory.
6.4	**(a)**	`cut -d":" -f1,3` specifies delimiter and requested field indices.
6.5	**(b)**	`tr 'a-z' 'A-Z'` translates all lowercase ASCII characters to uppercase.
6.6	**(c)**	The question mark (`?`) matches zero or one occurrence in ERE.
6.7	**(b)**	`tail -f` follows output as the file grows.
6.8	**(a)**	`sort -n -r -k2 -t","` sorts numerically, reversed, on column 2, with comma delimiter.
6.9	**(b)**	`grep -v` inverts the match condition.
6.10	**(b)**	`^$` asserts immediate start of line followed by end of line (empty line).

6.2 Technical Viva-Voce Questions & Conceptual Answers

Q1. Explain the syntactic differences between BRE and ERE regarding quantifiers and grouping.

Under **Basic Regular Expressions (BRE)**, metacharacters like `+`, `?`, `|`, `(`, and `{` are treated as literal characters unless escaped with a backslash (`\+`, `\?`, `\|`, `\( \)`, `\ \`). Under **Extended Regular Expressions (ERE)**, these characters are treated as active metacharacters natively: `+` (one or more), `?` (zero or one), `|` (alternation), `( )` (grouping), and `{m,n}` (repetition count).

Q2. Construct a pipeline that finds the 5 most frequent HTTP response status codes in a web server log.

Assuming standard Combined Log Format where the HTTP status code is column 9:

```
$ Interactive Terminal Verification

$ awk '{print $9}' access.log | sort | uniq -c | sort -rn | head -n 5
```

6.3 Lab Practical Exercises & Step-by-Step CLI Execution

Lab Practical Exercise 6.1 — Industrial Log Stream Analytics [Hands-On Implementation]

Objective: Process a simulated Apache web access log, extract client IP addresses, deduplicate, and sort by request volume.

```
$ Interactive Terminal Verification

$ cat << 'EOF' > web.log
192.168.1.10 - - [22/Oct/2026:08:00:01] "GET /index.html" 200 4500
10.0.0.5 - - [22/Oct/2026:08:00:02] "POST /api/login" 401 512
192.168.1.10 - - [22/Oct/2026:08:00:03] "GET /style.css" 200 1200
192.168.1.10 - - [22/Oct/2026:08:00:04] "GET /logo.png" 200 8900
10.0.0.5 - - [22/Oct/2026:08:00:05] "POST /api/login" 200 1024
EOF

$ cut -d' ' -f1 web.log | sort | uniq -c | sort -rn
      3 192.168.1.10
      2 10.0.0.5
```

Chapter 7

Unit 7: User & Group Administration Question Bank

7.1 Multiple Choice Questions (MCQs)

Q7.1. What numeric User ID (UID) is permanently assigned to the superuser (root)?
(a) `1000`
(b) `1`
(c) `0`
(d) `65534`

Q7.2. Which file contains encrypted user password hashes alongside aging parameters?
(a) `/etc/passwd`
(b) `/etc/shadow`
(c) `/etc/security`
(d) `/etc/group`

Q7.3. What does the field value 'x' in the second column of /etc/passwd indicate?
(a) The account has been deleted
(b) The account password is encrypted and stored in /etc/shadow
(c) The account has no password required
(d) The account is locked

Q7.4. Which command switch instructs useradd to automatically create the user's home directory?
(a) `-d`
(b) `-m`
(c) `-h`
(d) `-c`

Q7.5. Which command modifies password expiration parameters and aging policies?
(a) `usermod`
(b) `chage`
(c) `passwd -e`
(d) `expire`

Q7.6. Which command must be used to safely edit the /etc/sudoers file with syntax validation?
(a) `nano /etc/sudoers`
(b) `vim /etc/sudoers`
(c) `visudo`
(d) `editsudo`

Q7.7. What is the primary difference between executing 'su' versus 'su -'?
(a) `su` switches to root without asking for password
(b) `su -` provides a full login shell, loading target user environment and PATH
(c) `su` is deprecated in modern Linux

(d) `su -` restricts command execution to single-user mode

Q7.8. Which command deletes a user account while simultaneously removing their home directory and mail spool?

(a) `userdel -r username`
(b) `userdel -f username`
(c) `rmuser username`
(d) `deluser -all username`

Q7.9. In /etc/shadow, what hash algorithm is denoted by the password prefix 6?

(a) MD5
(b) Blowfish
(c) SHA-256
(d) SHA-512

Q7.10. Which command permanently locks a user account password by prepending an exclamation mark (!) in /etc/shadow?

(a) `usermod -L username`
(b) `useradd -lock username`
(c) `chage -d 0 username`
(d) `groupmod -L username`

Table 7.1: **Unit 7 MCQ Answer Key & Verification**

Question	Answer	Conceptual Core
7.1	**(c)**	The superuser account root is hardcoded to UID 0.
7.2	**(b)**	`/etc/shadow` stores cryptographic hashes, salt, and aging parameters.
7.3	**(b)**	The `x` placeholder signals that the hash resides in shadow file.
7.4	**(b)**	`useradd -m` provisions home directory from `/etc/skel`.
7.5	**(b)**	`chage` (change age) manipulates password rotation and warning windows.
7.6	**(c)**	`visudo` prevents syntax errors that could permanently lock out administrators.
7.7	**(b)**	`su -` loads login shell profiles, resetting environment variables and directory.
7.8	**(a)**	`userdel -r` recursively destroys user home directory and mail spool.
7.9	**(d)**	The identifier `$6$` indicates SHA-512 hashing with cryptographic salt.
7.10	**(a)**	`usermod -L` prepends `!` to password hash, blocking authentication.

7.2 Technical Viva-Voce Questions & Conceptual Answers

Q1. Explain the 7 fields of /etc/passwd with an exact example entry.

Entry: `student:x:1001:1001:CS Undergrad,Lab 4:/home/student:/bin/bash`

1. `student`: Username / login name.
2. `x`: Password placeholder indicating hash is in `/etc/shadow`.
3. `1001`: User ID (UID).
4. `1001`: Primary Group ID (GID).
5. `CS Undergrad,Lab 4`: GECOS comment field describing user.
6. `/home/student`: Absolute path to home directory.
7. `/bin/bash`: Login shell program spawned upon authentication.

7.3 Lab Practical Exercises & Step-by-Step CLI Execution

Lab Practical Exercise 7.1 — Provisioning Multi-Tier User Accounts [Hands-On Implementation]

Objective: Create an administrative user, configure password aging policies, append to the sudo group, and audit the shadow entry.

$ Interactive Terminal Verification

```
$ sudo useradd -m -s /bin/bash -c "DevOps Lead" devlead
$ sudo usermod -aG sudo devlead
$ sudo chage -M 60 -W 7 devlead
$ sudo chage -l devlead
Last password change                                    : Oct 22, 2026
Password expires                                        : Dec 21, 2026
Password inactive                                       : never
Account expires                                         : never
Minimum number of days between password change          : 0
Maximum number of days between password change          : 60
Number of days of warning before password expires       : 7
```

Chapter 8

Unit 8: Process Management & Signal Architecture Question Bank

8.1 Multiple Choice Questions (MCQs)

Q8.1. What is the process identifier (PID) of the systemd / init ancestor process in Linux?
(a) `0`
(b) `1`
(c) `-1`
(d) `100`

Q8.2. Which system call creates a duplicate child process with an identical address space?
(a) `fork()`
(b) `execve()`
(c) `clone()`
(d) `spawn()`

Q8.3. What is a Zombie (defunct) process?
(a) A process stuck in an infinite CPU loop
(b) A terminated process whose exit status has not yet been collected by its parent via wait()
(c) A background daemon executing without a controlling terminal
(d) A process whose memory has been paged out to swap

Q8.4. Why cannot a Zombie process be destroyed by sending: kill -9 <PID>?
(a) Zombie processes run in Ring 0 with elevated immunity
(b) The process has already terminated and released its memory; only its table entry remains
(c) The root user lacks permission to signal defunct processes
(d) `kill -9` only targets foreground jobs

Q8.5. Which POSIX signal cannot be intercepted, blocked, or caught by an application?
(a) `SIGINT (2)`
(b) `SIGTERM (15)`
(c) `SIGKILL (9)`
(d) `SIGHUP (1)`

Q8.6. Which keyboard shortcut sends the SIGTSTP signal, suspending a foreground process?
(a) `Ctrl+C`
(b) `Ctrl+Z`
(c) `Ctrl+\`
(d) `Ctrl+D`

Q8.7. Which command resumes a stopped process running in the background?
(a) `bg`
(b) `fg`
(c) `resume`

(d) `continue`

Q8.8. What does the STAT code 'D' in the output of the ps command indicate?

(a) Defunct / Zombie state
(b) Uninterruptible sleep (typically waiting on disk device I/O)
(c) Detached daemon
(d) Debug tracing state

Q8.9. Which command signals processes matching a specified pattern name?

(a) `killall`
(b) `pkill`
(c) `pskill`
(d) `term`

Q8.10. What is the function of the nohup command?

(a) Runs a process with maximum CPU scheduling priority
(b) Executes a command immune to SIGHUP terminal disconnect signals
(c) Prevents memory allocation errors
(d) Disables child subshell creation

Table 8.1: **Unit 8 MCQ Answer Key & Verification**

Question	Answer	Conceptual Core
8.1	**(b)**	The `systemd` or `init` process is the root of the process tree (PID 1).
8.2	**(a)**	`fork()` duplicates the calling process; `execve()` overwrites it with new code.
8.3	**(b)**	A zombie has exited, but remains in the process table until parent reaps its status.
8.4	**(b)**	Zombies are already deceased; signals cannot terminate dead processes.
8.5	**(c)**	`SIGKILL (9)` and `SIGSTOP (19)` cannot be trapped or ignored.
8.6	**(b)**	`Ctrl+Z` generates `SIGTSTP` (signal 20), placing the job into stopped state.
8.7	**(a)**	`bg %N` restarts a stopped job in background mode.
8.8	**(b)**	State `D` is uninterruptible sleep, awaiting hardware I/O completion.
8.9	**(b)**	`pkill` matches processes by regex pattern and user attributes.
8.10	**(b)**	`nohup` catches `SIGHUP`, ensuring background execution continues after logout.

8.2 Technical Viva-Voce Questions & Conceptual Answers

Q1. Explain the life cycle of a process from fork() through zombie state and reaping.

1. **Creation:** A parent process invokes `fork()`, creating an identical child process with a unique PID. 2. **Execution:** The child calls `execve()`, replacing its memory space with a new program image. 3. **Termination:** Upon completing execution, the process calls `exit(code)`. The kernel frees its virtual memory, open file descriptors, and heap pages. 4. **Zombie Phase:** The kernel retains a minimal entry in the process table holding its PID and exit status code. 5. **Reaping:** The parent invokes `wait()` or `waitpid()`, reading the child's exit status. Upon completion of `wait()`, the kernel removes the process table entry entirely.

8.3 Lab Practical Exercises & Step-by-Step CLI Execution

Lab Practical Exercise 8.1 — Terminal Job Control Workflow [Hands-On Implementation]

Objective: Launch a long-running simulation job, pause it via keyboard signal, verify background status, and return it to foreground.

```
$ Interactive Terminal Verification

$ python3 -c 'import time; [time.sleep(1) for _ in range(300)]'
^Z
[1]+  Stopped                 python3 -c '...'

$ jobs -l
[1]+ 24512 Stopped                 python3 -c '...'

$ bg %1
[1]+ python3 -c '...' &

$ jobs -l
[1]+ 24512 Running                 python3 -c '...' &

$ kill -15 24512
[1]+  Terminated              python3 -c '...'
```

Chapter 9

Unit 9: vi/vim Modal Text Editor Question Bank

9.1 Multiple Choice Questions (MCQs)

Q9.1. What is the default operating mode when vi opens a document?
(a) Insert Mode
(b) Command (Normal) Mode
(c) Ex (Last-Line) Mode
(d) Visual Mode

Q9.2. Which command key enters Insert mode by opening a new blank line BELOW the current cursor line?
(a) `i`
(b) `a`
(c) `o`
(d) `O`

Q9.3. Which key deletes the single character directly beneath the cursor in Command mode?
(a) `d`
(b) `x`
(c) `r`
(d) `c`

Q9.4. How does an engineer save buffer modifications and exit vi in Ex mode?
(a) `:q!`
(b) `:wq`
(c) `:w`
(d) `:exit!`

Q9.5. Which keystroke combination jumps the cursor directly to the absolute first line of the document?
(a) `G`
(b) `gg`
(c) `0`
(d) `^`

Q9.6. What does the vi command 5dd execute?
(a) Duplicates current line 5 times
(b) Deletes (cuts) 5 consecutive lines starting from cursor
(c) Jumps down 5 lines
(d) Searches for the number 5

Q9.7. Which Ex command globally replaces every occurrence of 'foo' with 'bar' across an entire file?
(a) `:s/foo/bar/g`

(b) `:%s/foo/bar/g`
(c) `:replace foo bar`
(d) `:g/foo/bar`

Q9.8. What does the dot (.) command do in vi Command mode?
(a) Inserts a period character
(b) Repeats the most recent text editing modification
(c) Moves cursor to line center
(d) Quits the editor

Q9.9. Which key sequence copies (yanks) the entire current line into the default register?
(a) `yy`
(b) `cc`
(c) `dd`
(d) `pp`

Q9.10. Which command key pastes the contents of the paste buffer AFTER or BELOW the active cursor?
(a) `p`
(b) `P`
(c) `v`
(d) `y`

Table 9.1: **Unit 9 MCQ Answer Key & Verification**

Question	Answer	Conceptual Core
9.1	**(b)**	vi defaults to Command / Normal mode for rapid navigation.
9.2	**(c)**	The lowercase `o` command opens a new line below and enters insert mode.
9.3	**(b)**	`x` deletes the character under the cursor.
9.4	**(b)**	`:wq` writes buffer to disk and quits the session.
9.5	**(b)**	`gg` navigates to line 1; `G` jumps to the last line.
9.6	**(b)**	Count prefixes multiply action: `5dd` cuts 5 lines.
9.7	**(b)**	`%` denotes full file range (1, $); `g` denotes global across each line.
9.8	**(b)**	The dot (`.`) command repeats the last editing change.
9.9	**(a)**	`yy` (yank line) copies line to register.
9.10	**(a)**	Lowercase `p` puts after/below; uppercase `P` puts before/above.

9.2 Technical Viva-Voce Questions & Conceptual Answers

Q1. Explain the modal editing paradigm of vi and why it increases productivity.

In modal editing, keys perform different operations depending on the current mode: 1. **Command Mode:** Keystrokes act as navigational vectors and text operators (`h,j,k,l`, `dd`, `yy`). Hands never leave the home row to reach for mouse or arrow keys. 2. **Insert Mode:** Keystrokes insert literal text. 3. **Ex Mode:** Accessed via colon (`:`) for file I/O, regex substitutions, and settings. Separating text navigation/manipulation from text insertion drastically accelerates high-velocity programming.

Chapter 10

Unit 10: Shell Programming & Automation Question Bank

10.1 Multiple Choice Questions (MCQs)

Q10.1. What is the primary role of the shebang line (#!/bin/bash) at the beginning of a script?

(a) It acts as a comment documenting author contact
(b) It instructs the kernel execve() loader which interpreter binary to spawn
(c) It sets root execution privileges
(d) It compiles the script into C bytecode

Q10.2. Which special variable holds the total count of positional arguments passed to a script?

(a) `$*`
(b) `$@`
(c) `$#`
(d) `$$`

Q10.3. Which test operator evaluates whether a specified path exists and is a REGULAR FILE?

(a) `-e`
(b) `-f`
(c) `-d`
(d) `-s`

Q10.4. What is the output of the Bash snippet: echo $((14 % 4))?

(a) `3.5`
(b) `2`
(c) `3`
(d) `0`

Q10.5. Which quoting style preserves the strict literal value of every enclosed character without exception?

(a) Double Quotes (`"..."`)
(b) Single Quotes (`'...'`)
(c) Backticks (`` `...` ``)
(d) Parentheses (`(...)`)

Q10.6. How are all elements of an indexed Bash array 'servers' expanded as separate words?

(a) `${servers}`
(b) `${servers[0]}`
(c) `${servers[@]}`
(d) `$#servers`

Q10.7. Which control flow keyword terminates an entire loop immediately?

(a) `continue`
(b) `break`

(c) `return`
(d) `exit`

Q10.8. What does the Bash defensive flag 'set -u' enforce?
(a) Executes commands in unprivileged mode
(b) Treats unset / uninitialized variables as fatal errors and exits immediately
(c) Disables Unicode formatting
(d) Forces unbuffered output

Q10.9. What is the correct syntax for testing whether integer variable $A is greater than or equal to $B?
(a) `[ $A >= $B ]`
(b) `[ $A -ge $B ]`
(c) `[ $A .GE. $B ]`
(d) `( $A => $B )`

Q10.10. Which command shifts positional parameters leftward ($2 becomes $1, $3 becomes $2)?
(a) `pop`
(b) `shift`
(c) `rotate`
(d) `next`

Table 10.1: **Unit 10 MCQ Answer Key & Verification**

Question	Answer	Conceptual Core
10.1	**(b)**	The kernel extracts interpreter binary path from the magic number `#!`.
10.2	**(c)**	`$#` stores the integer count of arguments passed.
10.3	**(b)**	`-f` checks whether target exists and is a regular file.
10.4	**(b)**	`%`: modulo operator returns remainder of division ($14 \pmod 4 = 2$).
10.5	**(b)**	Single quotes prevent all variable expansions and escapes.
10.6	**(c)**	`${arr[@]}` expands all elements safely preserving whitespace.
10.7	**(b)**	`break` breaks out of loops; `continue` skips to next iteration.
10.8	**(b)**	`set -u` prevents catastrophic bugs caused by misspelled variable names.
10.9	**(b)**	Relational operator for greater-than-or-equal in `[ ... ]` is `-ge`.
10.10	**(b)**	`shift` discards `$1` and shifts arguments to the left.

10.2 Shell Scripting & DevOps Automation Challenges

DevOps & Shell Scripting Challenge 10.1 — Automated Server Log Archival Engine [Industrial Scenario]

Scenario: Design a complete, production-grade Bash script that parses a log directory, compresses files older than 7 days into a dated tarball, deletes original logs upon successful compression, and logs all operations with timestamps.

Production Bash Script

```
#!/bin/bash
set -euo pipefail

LOG_DIR="/var/log/app"
BACKUP_DIR="/backup/logs"
DATE_STAMP=$(date +%Y%m%d_%H%M%S)
ARCHIVE_NAME="logs_archive_${DATE_STAMP}.tar.gz"

# 1. Input Validation
if [ ! -d "$LOG_DIR" ]; then
    echo "[ERROR] Log directory $LOG_DIR does not exist." >&2
    exit 1
fi
mkdir -p "$BACKUP_DIR"

# 2. Identify and Archive Old Logs
OLD_FILES=$(find "$LOG_DIR" -type f -name "*.log" -mtime +7)

if [ -z "$OLD_FILES" ]; then
    echo "[INFO] No log files older than 7 days identified. Exiting."
    exit 0
fi

echo "[INFO] Commencing archival into $BACKUP_DIR/$ARCHIVE_NAME..."
tar -czf "$BACKUP_DIR/$ARCHIVE_NAME" -T <(echo "$OLD_FILES")

# 3. Verify Archive Integrity Before Deletion
if [ -s "$BACKUP_DIR/$ARCHIVE_NAME" ]; then
    echo "[INFO] Archive generated successfully. Removing archived logs..."
    echo "$OLD_FILES" | xargs rm -f
    echo "[SUCCESS] Archival cycle concluded successfully."
else
    echo "[FATAL] Archive file was empty! Aborting deletion." >&2
    exit 2
fi
```

Chapter 11

Unit 11: Development Tools & Git Version Control Question Bank

11.1 Multiple Choice Questions (MCQs)

Q11.1. Which command initializes a new Git repository in the current working directory?

(a) `git start`
(b) `git init`
(c) `git create`
(d) `git new`

Q11.2. Where does Git store its complete object database, references, and configuration?

(a) In `/etc/git.conf`
(b) In the hidden `.git/` directory in the project root
(c) On the remote GitHub server exclusively
(d) In `/var/log/git`

Q11.3. Which Git command moves modified files from the Working Directory into the Staging Area (Index)?

(a) `git stage`
(b) `git commit`
(c) `git add`
(d) `git push`

Q11.4. What immutable object in the Git object database represents a snapshot of directory contents?

(a) Blob
(b) Tree
(c) Commit
(d) Tag

Q11.5. What is a branch in Git's internal architecture?

(a) A complete physical copy of all source code files
(b) A lightweight 41-byte plain-text pointer storing a 40-character commit SHA hash
(c) A database table holding diff deltas
(d) A remote server URL

Q11.6. Which command creates a new branch named 'feature' and immediately checks it out?

(a) `git branch -new feature`
(b) `git switch -c feature`
(c) `git merge feature`
(d) `git pull feature`

Q11.7. What occurs during a Fast-Forward merge in Git?

(a) A new merge commit with two parents is generated

(b) Git simply moves the active branch pointer forward to the tip of the merged branch
(c) Git deletes the merged branch permanently
(d) Git rejects the merge due to conflict

Q11.8. Which command downloads remote repository branches and commits without merging them into active branches?

(a) `git pull`
(b) `git fetch`
(c) `git clone`
(d) `git sync`

Q11.9. What do the conflict markers «««< HEAD denote during a merge conflict?

(a) The beginning of the changes on the incoming branch
(b) The beginning of conflicting changes on the currently checked-out branch
(c) The commit hash of the common ancestor
(d) A fatal corruption in the Git index

Q11.10. Which command line tool provides native terminal integration for managing GitHub PRs, issues, and releases?

(a) `git-hub`
(b) `gh`
(c) `hub-cli`
(d) `git-pr`

Table 11.1: **Unit 11 MCQ Answer Key & Verification**

Question	Answer	Conceptual Core
11.1	**(b)**	`git init` provisions the local repository structure.
11.2	**(b)**	All history, blobs, and branches reside inside the local `.git/` folder.
11.3	**(c)**	`git add` stages modified files into the index.
11.4	**(b)**	Trees represent directory hierarchies; Blobs store pure file content.
11.5	**(b)**	A Git branch is merely a movable pointer pointing to a commit object.
11.6	**(b)**	`git switch -c` (or `git checkout -b`) creates and switches branches.
11.7	**(b)**	Fast-Forward advances the branch pointer without creating a merge commit.
11.8	**(b)**	`git fetch` downloads remote references without altering the local working tree.
11.9	**(b)**	««««< HEAD identifies lines from the current branch.
11.10	**(b)**	`gh` is the official GitHub command-line interface.

11.2 Technical Viva-Voce Questions & Conceptual Answers

Q1. Explain the Three-Tree Architecture of Git and trace a file change across all three.

Git manages files across three logical domains: 1. **Working Directory:** The physical files on disk that the developer edits directly. 2. **Staging Area (Index):** A binary file at `.git/index` that prepares the snapshot for the next commit. Executing `git add <file>` computes the SHA hash of the file, writes a blob object into `.git/objects/`, and registers the path in the index. 3. **Repository (HEAD):** The permanent history of committed snapshots. Executing `git commit` creates a tree object from the index, builds a commit object pointing to that tree, and advances the branch pointer and HEAD to the new commit.

11.3 Lab Practical Exercises & Step-by-Step CLI Execution

Lab Practical Exercise 11.1 — Git Branching, Conflict Simulation and Resolution [Hands-On Implementation]

Objective: Create divergent commits on two branches modifying the same file line, induce a merge conflict, and resolve it manually.

$ Interactive Terminal Verification

```
# 1. Initialize repository and make initial commit:
$ git init demo-repo && cd demo-repo
$ echo "TIMEOUT = 30" > config.py
$ git add config.py && git commit -m "Initial commit"

# 2. Create feature branch and modify timeout:
$ git switch -c feature-perf
$ echo "TIMEOUT = 10" > config.py
$ git commit -am "Optimize timeout for performance"

# 3. Switch back to main and make conflicting change:
$ git switch main
$ echo "TIMEOUT = 60" > config.py
$ git commit -am "Increase timeout for resilience"

# 4. Attempt merge to trigger conflict:
$ git merge feature-perf
Auto-merging config.py
CONFLICT (content): Merge conflict in config.py
Automatic merge failed; fix conflicts and then commit the result.

# 5. Resolve conflict and complete merge:
$ cat << 'EOF' > config.py
TIMEOUT = 45  # Compromise resolved value
EOF
$ git add config.py
$ git commit -m "Merge branch feature-perf: resolved timeout setting"
[main 7f2a10b] Merge branch feature-perf: resolved timeout setting
```